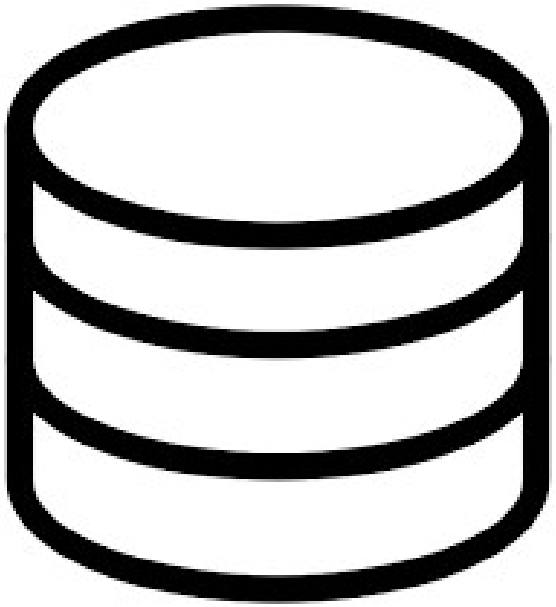
## En database, en struktureret datamængde

entiteter & attributter
datamodel & E/R-diagram

relationstype & kardinalitet

normaliseringsformer &
dataintegritet

SQL



# En database = en struktureret datamængde

En bruger kan manipulere den via en grænseflade – i det basale brugsmønster **CRUD**: create, read, update & delete

CRUD-handlingerne igennem grænsefladen foretages af et *manipulation-sprog* (DML), fx SQL, som jo netop står for **”Structured Query Language”**

Selve databasen skal dog formgives:
databasens elementer og deres relationer skal formgives i en **datamodel.**

**Datamodellen** kan gribes an som en **E/R-model**, der står for ”Entity/Relationship”:
- Modellen opstilles ud fra 'entiteter' og deres forhold (i en relationel database).
- entiteterne har en række attributter af forskellig art med en værdi inden for en vis værdimængde,
- fx entiteten 'Ansat' kunne have værdien 'Køn', og da mængden 'Mand' eller 'Kvinde’.

# En database = en struktureret datamængde

En bruger kan manipulere den via en grænseflade – i det basale brugsmønster **CRUD**: create, read, update & delete

**Læs om databaser i teksten OmDatabaser:**

Hvad menes der med primær og kandidatnøgler?

Hvad er entiteter & attributter?

Hvilke slags relationer imellem entiteter findes der?

Forklar diagrammet på side 9?

# En database = en struktureret datamængde

Et DBMS betyder et database management system:
Fx SAP, MySQL eller ACCES (microsoft)

Når ens data og tabeller vokser ud over, hvad man kan overskue i et regneark, bruger man gerne databaser.

Hvornår i dagligdagen benytter man databaser?
Fx biblioteker, netbank, borger.dk, lectio, statistik-sider…

**Normalisering** – at have en datamodel, som sikrer data-integritet – dvs. korrekte data og rigtige svar.
At datamodellen ikke giver anledning til forkerte svar efter fx opdatering eller sletning af data.
Normalisering af databasen handler derfor om at give den et godt design!
Dataen skal være entydig og konsistent, dvs. at samme data bør helst ikke optræde flere steder (redundans) eller data bør ikke være i modstrid med anden data, altså mangle sammenhæng.

Omvendt kan en stringent normalisering tynge applikationens ydeevne, idet normalformerne ofte vil betyde flere tabeller og dermed flere forespørgsler. Derfor kan normalisering opvejes imod eventuelt tab af væsentlig performance.

Man kan tale om **3 normalformer** for en tabel eller database.
Hvis alle tabellerne er i en normalform, er selve databasen det også. Det minimerer risikoen for fejl!

**Redundans** betyder, at vi har de samme data stående flere steder databasen, hvilket vi gerne vil undgå. Hvis vi eksempelvis, hver gang vi får en ny ordre fra den samme kunde, skal skrive kundens navn og adresse, får vi efterhånden disse oplysninger til at stå i mange poster. Hvis vi så skriver kundens navn og adresse lidt forskelligt i de forskellige ordrer, har vi **inkonsistente data**. Inkonsistente data betyder, at data, der burde være ens, ikke er det.

| OrdreID | Pris | Navn | Gade | Postnummer |
|---|---|---|---|---|
| 120 | 2500 | Bo Hansen | Æblevænget 2 | 2900 |
| 121 | 800 | Dorthe Rasmussen | Slingre Allé 10 | 8000 |
| 122 | 7500 | Bo Hansen | Pærevænget 2 | 2900 |

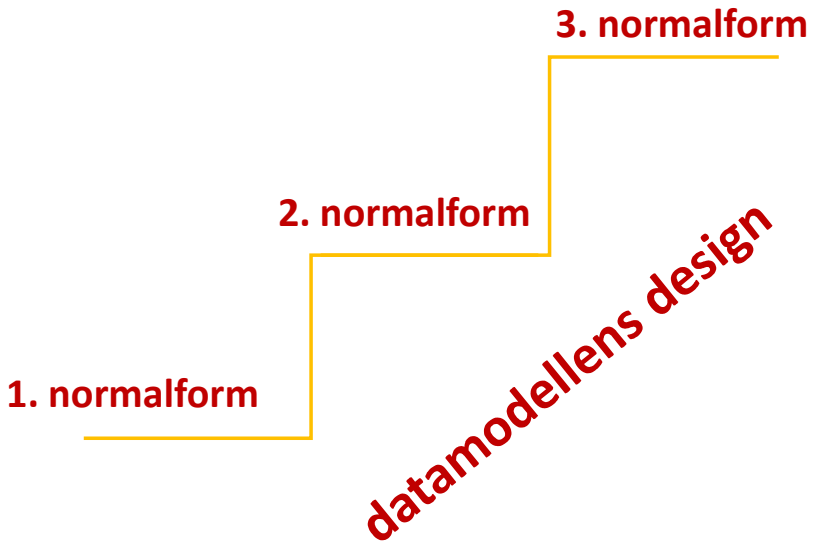
# **Normalisering** – bedre design af **DATAMODELLEN**

Hvis alle tabellerne er i en **normalform**, er selve databasen det også. Det minimerer risikoen for fejl!

For at en tabel eller database kan være i 2. normalform, skal den først være i 1. normalform, og det samme med 3. normalform: For at den kan være i 3. normalform, skal den først være 2. normalform!

**Modellering af data:**

- Skriv ord fra domænet op
- Find bud på tabeller/entitetsklasser, ofte via navneord
- Find relationer, ofte via udsagnsord
- Find attributter, egenskab ved eller oplysning om en entitet



## Første normalform: nøglefelt til den specifikke post + atomare værdier

Tabellen har en primærnøgle!
(primærnøglen er den specifikke og unikke værdi, hvormed man kan identificere en specifik række/post i tabellen)
Tabellens attributter er simple én-værdi-attributter, dvs. de er ikke sammensatte. Og de indeholder samme type data.

En én-værdi-attribut kunne være fx ”alder” – ”21”, (atomar værdi)
mens et sammensat datafelt i tabellen kunne være ”kæledyr” – ”kat guldfisk hamster”.
Sammensatte datafelter er bøvlede at læse fra og opdatere – giver anledning til fejl. Noget bøvl.
Men primærnøglen kan godt være en sammensat værdi {a, b},
fx i en **Danmarksmester-tabel {sportsgren, årstal}** – med attributten ‘klub’: kombinationen af sportsgren og årstal er nøglen!

Og alle poster er lige lange – har samme antal felter i rækken.
Nogle kan eventuelt have ‘null’ som værdi, altså værdien for at der ikke er en værdi.

Nedenfor en **Lærer-tabel** med 2 poster, med 5 attributter i hver, hvor feltet ‘Initialer’ er primærnøglen (eneste kandidatnøgle)

| Initialer | Køn | Fag1 | Fag2 | Fag3 |
|---|---|---|---|---|
| obr | Mand | Idræt | Biologi | Kemi |
| lit | Kvinde | Matematik | Fysik | null |

**Primærnøglen** identificerer den specifikke post – som informerer om en lærer – køn samt de fag, læreren underviser i.
Alle felter er udgjort af kun én værdi, og alle poster er lige lange.

## Anden normalform: øvrige felter afhænger af hele den sammensatte nøgle

Tabellen indeholder IKKE et sammensat nøglefelt ***eller***
de felter, som ikke er en del af den primære nøgle, må ikke kun *afhænge* af en del af denne.
De skal netop *afhænge* af hele den sammensatte nøgle, og ikke kun en del af denne…

Hvis en sammensat primærnøgle fx består af Navn, Gade & Postnummer, så vil feltet Bynavn kun *afhænge* af Postnummer og ikke hele nøglen.
Derfor er denne sammensatte nøgle ikke hensigtsmæssig og ikke i Anden normalform!
Det giver r**edundans** og dermed øget risiko for inkonsistente data, altså forkerte data = forkerte svar ☹

**Udvalgsmøde-tabellen:** Tidspunkt og udvalg er her sammensat primærnøgle – men mødelokale *afhænger* kun af udvalg, hvis vi siger, at udvalget netop altid holder møder i samme lokale. Mødelokale er bestemt af Udvalg.
Derfor er der ingen grund til at skulle taste det ind igen og igen, da det øger risikoen for forkerte data….
Her er mødelokalet, som jo kun afhænger af udvalg, tastet ind to gange – hvilken er den rigtige ?

| Tidspunkt | Udvalg | Mødelokale |
|---|---|---|
| 11:00 | Festudvalget | 1.15 |
| 15:30 | Festudvalget | 1.25 |
| 17:15 | Skoleavisudvalget | 1.35 |

**FEJL!**
**- hvad er det rigtige?**

Derfor bedre at rykke det ud i selvstændig tabel, hvor Udvalg er nøgle til Mødelokale som datafelt:
På den måde skal man netop skrive mødelokalet, som hører til udvalget, kun ÈT sted.

## Anden normalform: øvrige felter afhænger af hele den sammensatte nøgle

Tabellen indeholder IKKE et sammensat nøglefelt ***eller***
de felter, som ikke er en del af den primære nøgle, må ikke kun *afhænge* af en del af denne.
De skal netop *afhænge* af hele den sammensatte nøgle, og ikke kun en del af denne…

**BookingBur-tabel** til at holde styr på brugen af de mange bure i en given periode
Burnummer og indlogerings-dato er sammensat primærnøgle – man skal ikke kunne bruge samme bur i samme periode

**Hvorfor ikke i 2NF?**

| Bur | Indlogering | DyrID | AntalDage | Burtype |
|---|---|---|---|---|
| 2 | 24-12-2023 | 282 | 4 | Udenfor |
| 4 | 10-07-2024 | 301 | 14 | Indenfor |
| 11 | 01-04-2024 | 529 | 1 | Indenfor-udenfor |
| 4 | 03-08-2024 | 7 | 10 | Udenfor |

# Tredie normalform: de øvrige felter må ikke være indbyrdes afhængige

De felter, som ikke er del af primærnøglen, må ikke være indbyrdes afhængige – ét felt må ikke bestemme et andet.
De skal være *uafhængige* af hinanden for at tabellen/databasen er i 3. normalform.

Hvis en tabel har attributterne Postnummer og By, så er By-værdien afhængig af Postnummer-værdien:
**Postnummer definerer By** (By kan have flere postnumre, spørg bare i københavn ☺)

Dette strider imod **3F**, og kan være en kilde til fejl i forskellige poster, idet samme postnummer ved en fejl kan associere til forskellige byer: Hvilken er så den rigtige?

Hvis ét felt er afhængig af et andet, skal det rykkes ud i en selvstændig tabel, fx **Postnummer-tabel:** 3000 Helsingør

| Festival | Dato | Pris | Postnummer | By |
|---|---|---|---|---|
| Roskilde | 01-07 | 1200 | 3000 | Helsingør |
| Copenhell | 19-06 | 1000 | 1432 | København |
| Smukfest | 04-08 | 900 | 2800 | Helsingør |

FEJL!

# Første normalform: Er tabellen i 1NF?

- navn er primærnøglen

| Navn | Art | Alder | Vægt | Foder | Køn |
|---|---|---|---|---|---|
| Kitkat | Kat | 5 | 3.5 | KatteLux | Fem |
| Flemming | Hund | 9 | 12 | DoggieLux | Mask |
| Mads | Ged | 12 | 14 | CabraDiscount | Mask |
| Slytherin | Cobra | 4 | 3 | KyllingeNuggetØko | Fem |
| Kithund | Hund | 11 | 7 | DoggieLuxØko | Fem |

# Anden normalform: Er tabellen i 2NF?

Tabellen indeholder IKKE et sammensat nøglefelt ***eller***

de felter, som ikke er en del af den primære nøgle, må ikke kun *afhænge* af en del af denne.
De skal netop *afhænge* af hele den sammensatte nøgle, og ikke kun en del af denne…

**Nedenfor en Medicin-tabel på det billige pensionat FerieDyrSjælland:**

Begrund at tabellen er **i 1NF**, men *ikke* i **2NF**
**-** dyreID og behandlingsDato er her sammensat primærnøgle

| dyreID | behandlingDato | medicin | dosis | ejer |
|---|---|---|---|---|
| 245 | 01-08-24 | Sovemiddel | 5 ml | 342 |
| 333 | 07-09-24 | Antibiotika | 12 mg | 110 |
| 211 | 19-09-24 | Panodil | 10 mg | 321 |
| 45 | 01-10-24 | Loppekur | 5 ml | 55 |

## Tredie normalform: de øvrige felter må ikke være indbyrdes afhængige

De felter, som ikke er del af primærnøglen, må ikke være indbyrdes afhængige – ét felt må ikke bestemme et andet.
De skal være uafhængige af hinanden for at tabellen/databasen er i 3. normalform.

**Nedenfor en Dyre-tabel på det billige pensionat FerieDyrSjælland:**

Begrund at tabellen er i både 1NF og 2NF, men *ikke* i **3NF**
- En given art spiser og får altid *samme* kost ☹

| DyreID | Art | Alder | Vægt | Foder | Køn |
|---|---|---|---|---|---|
| 5 | Kat | 5 | 3.5 | KatteLux | Fem |
| 6 | Hund | 9 | 12 | DoggieLux | Mask |
| 7 | Ged | 12 | 14 | CabraDiscount | Mask |
| 8 | Cobra | 4 | 3 | KyllingeNuggetØko | Fem |
| 9 | Hund | 11 | 7 | DoggieLuxØko | Fem |

## Tredie normalform: de øvrige felter må ikke være indbyrdes afhængige

De felter, som ikke er del af primærnøglen, må ikke være indbyrdes afhængige – ét felt må ikke bestemme et andet. De skal være uafhængige af hinanden for at tabellen/databasen er i 3. normalform.

**Nedenfor en Dyre-tabel på pensionatet FerieDyrSjælland:**
DyreID er primærnøglen (eneste mulige), og alle felter har atomare værdier, **altså 1.normalform.**
Den er også i **2.normalform**, da der ikke er en sammensat nøgle. Hvis nøglen havde været sammensat, så skulle de øvrige felter afhænge af hele den sammensatte nøgle og ikke kun dele af den. Men er den i **3. normalform?**

Nej – ikke hvis værdien af 'Art' bestemmer værdien af 'Foder', altså hvis 'Kat' altid skal fodres med KatteLux-mad.

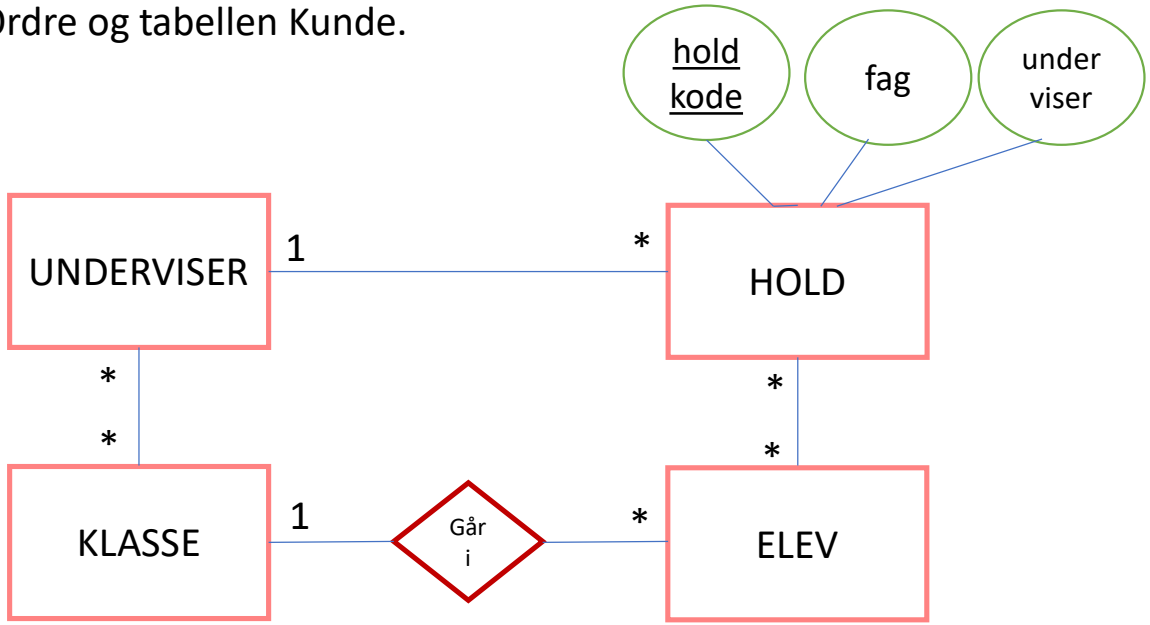
Løsning: Det bør derfor tages ud i selvstændig Tabel, hvor Art er nøglen og Foder attribut. Minimerer risikoen for fejl…

| DyreID | Art | Alder | Vægt | Foder | Køn |
|---|---|---|---|---|---|
| 5 | Kat | 5 | 3.5 | KatteLux | Fem |
| 6 | Hund | 9 | 12 | DoggieLux | Mask FEJL! |
| 7 | Ged | 12 | 14 | CabraDiscount | Mask |
| 8 | Cobra | 4 | 3 | KyllingeNuggetØko | Fem |
| 9 | Hund | 11 | 7 | DoggieLuxØko | Fem FEJL! |

# E/R-DIAGRAM = entitets/relations-diagram

Hvilke sammenhænge er der mellem de forskellige tabeller i databasen ? Det kan sættes på diagram!
Ud fra diagrammet kan man få et billede af den **datamodel**, som den relationelle database følger eller skal følge…
En tabel er en **entitetsklasse (en klasse af dimser)** – og en konkret post i tabellen en **entitet (en dims).**
**Relationer** er de sammenhænge der er mellem tabellerne, og dermed de specifikke entiteter…

Diagrammet fortæller om hvilke tabeller, som er forbundne, og hvad der kendetegner deres relation:
Fx kan en 'Ordre' kun have én 'Kunde', mens 'Kunde' kan have mange 'Ordre', altså en **1-mange-relation** imellem tabellen Ordre og tabellen Kunde.



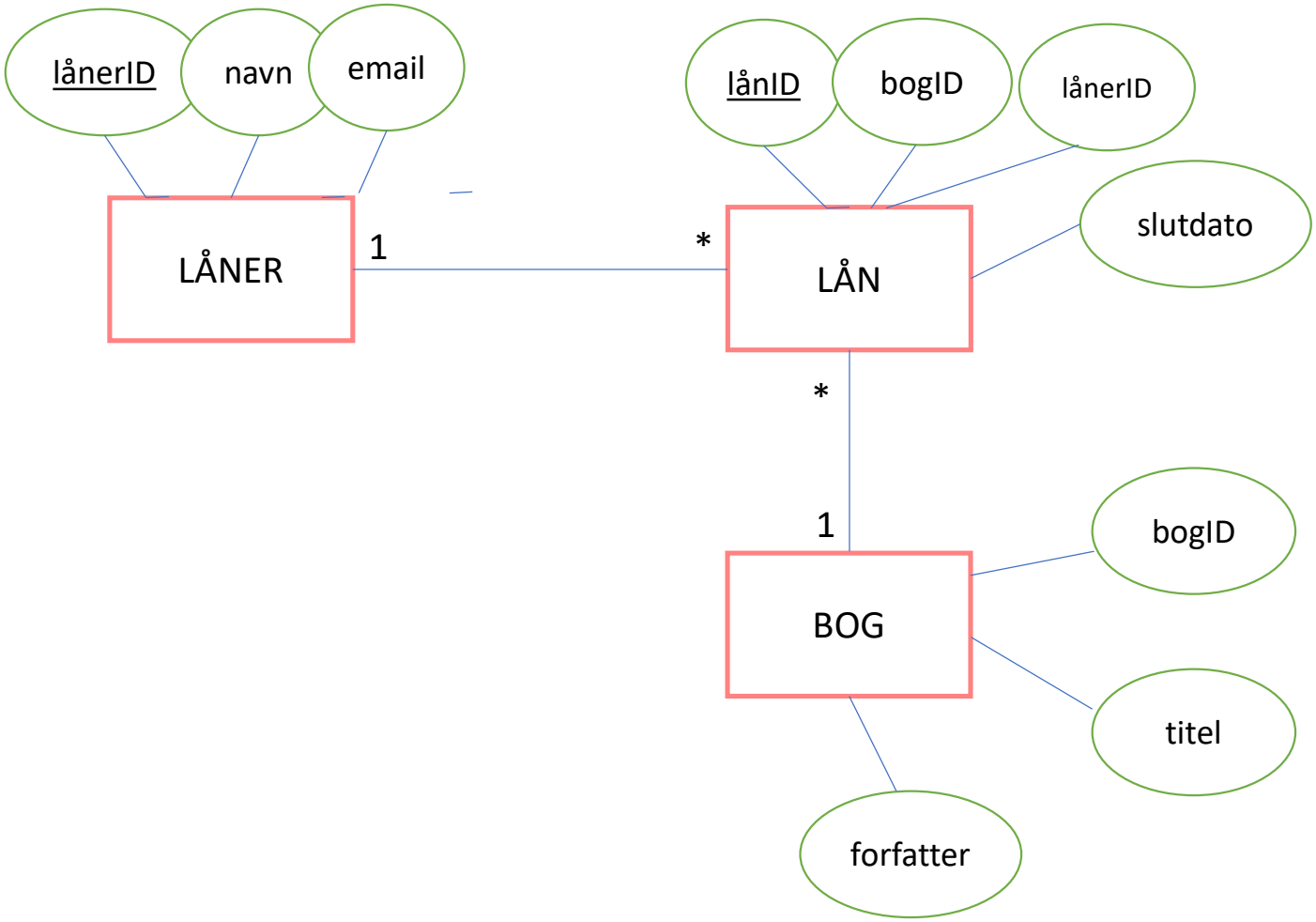

- relation kan beskrives i diamant imellem kasserne

Et hold har én underviser
Et hold har mange elever

En elev har én klasse
En elev har mange hold

En underviser har mange hold
En underviser har mange klasser

En klasse har mange undervisere
En klasse har mange elever

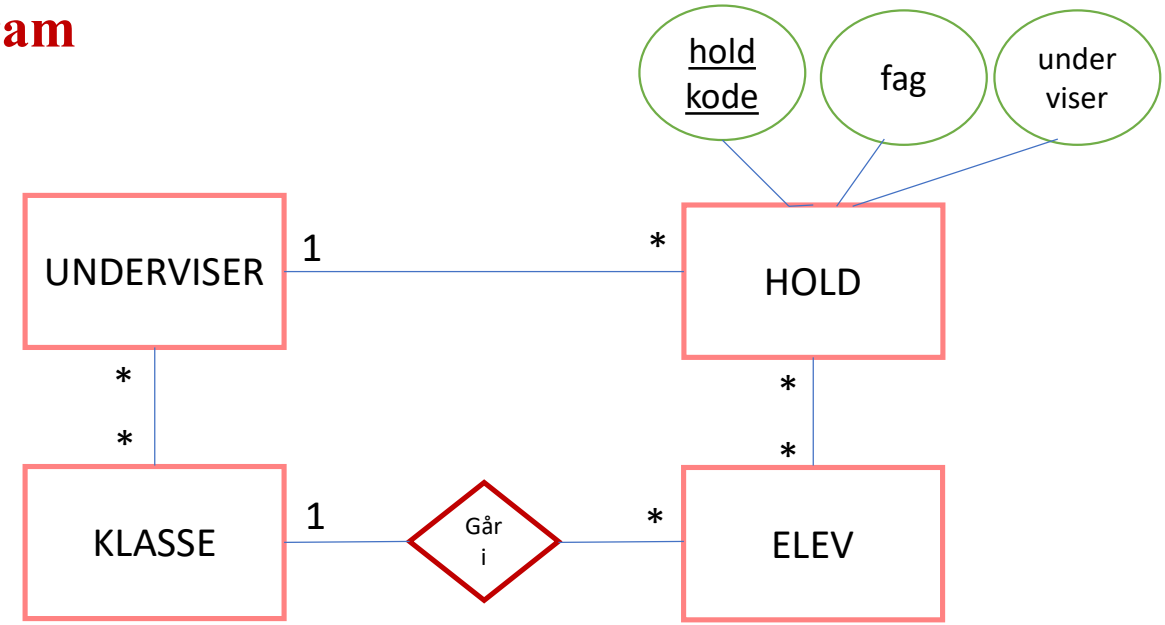



En låner har mange lån

Et lån har en låner
Et lån har en bog

En bog har mange lån

# E/R-DIAGRAM = entitets/relations-diagram




Et hold har én underviser,
En underviser har mange hold:
En 1-til-mange relation *fra* hold *til* underviser

Et hold har mange elever
En elev er på mange hold
En mange-til-mange relation ml. hold & elever

En elev går i én klasse
En klasse mange elever
En 1-til-mange relation *fra elev til* klasse

1-til-1 relation er mere sjældne, eksempel:
én rektor er på et gymnasium,
ét gymnasium har én rektor

# RELATIONERNE I DATABASEN

# HVORDAN IMPLEMENTERE?

**1-mange relation mellem A til B (eller fra B til A)**

- tabel A's primærnøgle kan være fremmednøgle i tabel B og
- række i B er dermed forbundet til kun én række i A
- mens én række i A kan være forbundet til mange rækker i B

**mange-til-mange relation mellem A & B**

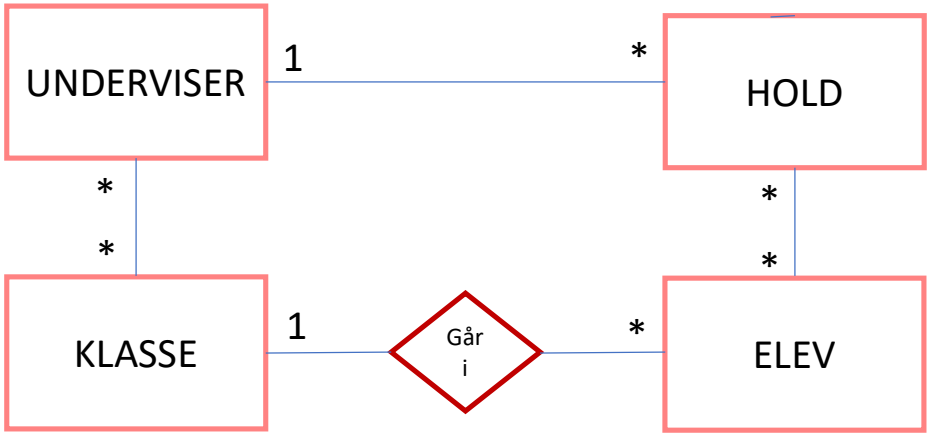
- Dette realiseres via en 'junction tabel', eller 'associativ tabel', altså netop en tredje AB-tabel, hvori A og B mødes ved, at primærnøgle fra både A og B indgår som fremmednøgler i AB.

**1-1 relation mellem A & B (sjælden!)**

- en række i Tabel A er forbundet til kun en række i Tabel B,
- og omvendt
- Ofte realiseret ved at den ene tabel's primærnøgle er fremmednøgle i den anden tabel.

Et hold har én underviser & en underviser har mange hold:
En 1-til-mange relation *fra* hold *til* underviser

Et hold har mange elever & en elev er på mange hold
En mange-til-mange relation ml. hold & elever

En elev går i én klasse & en klasse mange elever
En 1-til-mange relation *fra elev til* klasse

1-til-1 relation er mere sjældne, eksempel:
én rektor er på et gymnasium, ét gymnasium har én rektor